

Relazione sul Class Diagram UML

1. Introduzione

Il presente documento descrive il class diagram UML realizzato per il sistema informativo di gestione ospedaliera.

2. Descrizione delle classi

2.1 Utente

La classe **Utente** rappresenta il concetto generale di utente del sistema. Ogni utente è necessariamente o un Amministratore o un Medico. Questa scelta progettuale segue il principio di generalizzazione, estraendo gli attributi comuni alle due tipologie in un'unica superclasse.

Attributi:

- **username** — il nome utente con cui si accede al sistema, univoco per ogni utente.
- **password** — la password associata all'account, anch'essa univoca.

Metodi:

- **login()** — verifica le credenziali inserite dall'utente e restituisce true se l'accesso è avvenuto con successo, false altrimenti.
-

2.2 Admin

La classe **Admin** è una sottoclasse di **Utente** e rappresenta il personale amministrativo della struttura ospedaliera. Eredita username, password e il metodo login() dalla superclasse.

Metodi:

- **gestisciPazienti()** — permette di inserire e modificare l'anagrafica dei pazienti nel sistema.
- **gestisciRicoveri()** — consente di registrare nuovi ricoveri, assegnando un paziente a un letto specifico per un determinato periodo. Internamente si avvale del controllo di sovrapposizione per garantire l'integrità dei dati.

- `elencoSostituzioni()` — genera la lista dei medici sostitutivi disponibili quando un medico risulta assente per malattia. I candidati vengono selezionati tra i medici dello stesso reparto che non hanno turni o prestazioni nelle fasce orarie scoperte.
-

2.3 Medico

La classe `Medico` è anch'essa sottoclasse di `Utente` e rappresenta il personale medico della struttura. I medici afferiscono a un singolo reparto e svolgono turni lavorativi settimanali. Durante i ricoveri eseguono prestazioni, ovvero visite o interventi chirurgici.

Attributi:

- `matricola` — codice identificativo univoco del medico all'interno del sistema.

Metodi:

- `agendaGiornaliera()` — restituisce l'elenco delle prestazioni programmate per il giorno corrente.
 - `agendaSettimanale()` — restituisce l'elenco delle prestazioni per la settimana in corso.
 - `registraPrestazione()` — permette al medico di registrare una nuova prestazione, verificando che ricada all'interno di un turno lavorativo e che non si sovrapponga temporalmente a un'altra prestazione già eseguita dallo stesso medico.
 - `disponibilità()` — verifica se il medico è disponibile in una determinata fascia oraria. Viene utilizzato da `elencoSostituzioni()` dell'Amministratore per individuare i candidati sostituti.
-

2.4 AssenzaMedico

La classe `AssenzaMedico` è stata introdotta per soddisfare il requisito aggiuntivo previsto per i gruppi da tre persone. Modella il periodo di malattia di un medico, consentendo al sistema di individuare automaticamente i turni e le prestazioni rimaste scoperte e di suggerire medici sostitutivi.

Attributi:

- `dataInizio` — data di inizio del periodo di assenza.
- `dataFine` — data di fine del periodo di assenza.

Metodi:

- `getTurniScoperti()` — restituisce l'elenco dei turni e delle prestazioni che rimangono privi di copertura nel periodo di assenza, fornendo all'Amministratore le informazioni necessarie per trovare un sostituto.

2.5 Reparto

La classe `Reparto` rappresenta un reparto della struttura ospedaliera. Ogni reparto è composto da più stanze ed è il contesto organizzativo a cui afferiscono i medici.

Attributi:

- `ID_reparto` — identificativo univoco del reparto.
- `nome` — denominazione del reparto.

Metodi:

- `cercaLettiLiberi()` — restituisce l'elenco dei letti attualmente disponibili nel reparto.

2.6 Stanza

La classe `Stanza` rappresenta una stanza all'interno di un reparto. Ogni stanza ospita un numero definito di letti.

Attributi:

- `numero` — identificativo della stanza all'interno del reparto.
- `capienza` — numero massimo di letti che la stanza può ospitare.

Metodi:

- `getLettiDisponibili()` — restituisce i letti della stanza che non risultano attualmente occupati. Viene invocato da `cercaLettiLiberi()` del `Reparto`.

2.7 Letto

La classe `Letto` rappresenta un singolo posto letto all'interno di una stanza. Il codice del letto è univoco nell'intero ospedale, non solo all'interno del reparto o della stanza.

Attributi:

- `ID_letto` — codice univoco del letto a livello di intero ospedale.
- `occupato` — indica se il letto è attualmente occupato da un paziente. Questo attributo viene visualizzato in rosso nell'interfaccia grafica per segnalare l'indisponibilità del posto letto in tempo reale.

2.8 Paziente

La classe `Paziente` rappresenta un paziente registrato nel sistema. Contiene i dati anagrafici minimi necessari per la gestione ospedaliera.

Attributi:

- `codiceFiscale` — identificativo univoco del paziente.
- `nome` — nome del paziente.
- `cognome` — cognome del paziente.

Metodi:

- `getInfoBase()` — restituisce le informazioni anagrafiche essenziali del paziente, utile ad esempio per visualizzare l'elenco dei pazienti in scadenza di dimissione.
-

2.9 Ricovero

La classe `Ricovero` rappresenta l'associazione tra un paziente e un letto per un determinato periodo di tempo. È il fulcro della gestione dei posti letto e contiene le informazioni sia sulle dimissioni previste che su quelle effettive, che possono differire.

Attributi:

- `dataInizio` — data di inizio del ricovero.
- `oraInizio` — ora di inizio del ricovero.
- `dataDimissionePrevista` — data di dimissione inizialmente pianificata.
- `oraDimissionePrevista` — ora di dimissione inizialmente pianificata.
- `dataDimissioneEffettiva` — data di dimissione reale, compilata al momento della dimissione.
- `oraDimissioneEffettiva` — ora di dimissione reale.

Metodi:

- `checkSovrapposizione()` — verifica che il letto selezionato non sia già occupato nell'intervallo di tempo del nuovo ricovero. La traccia richiede che questo controllo sia categorico, impedendo qualsiasi inserimento o modifica che generi una sovrapposizione.
 - `inCorso()` — verifica se il ricovero è ancora attivo, ovvero se il paziente non è ancora stato dimesso. Viene utilizzato per produrre l'elenco dei pazienti in scadenza di dimissione nella giornata odierna o in una data specifica.
-

2.10 Turno

La classe `Turno` rappresenta una fascia oraria lavorativa settimanale assegnata a un medico. I turni definiscono il periodo in cui il medico è presente e operativo nel reparto.

Attributi:

- `ID_Turno` — identificativo univoco del turno.
- `oraInizio` — ora di inizio della fascia lavorativa.
- `oraFine` — ora di fine della fascia lavorativa.
- `giornoSettimana` — giorno della settimana a cui il turno fa riferimento.

Metodi:

- `copreFascia()` — verifica se una determinata ora ricade all'interno della fascia oraria del turno. Viene invocato prima della registrazione di una prestazione per garantire che il medico stia operando durante un suo turno lavorativo.
-

2.11 Prestazione

La classe `Prestazione` rappresenta un intervento medico eseguito da un medico su un paziente ricoverato. Può essere una visita o un intervento chirurgico. La traccia richiede che ogni prestazione sia associata a un turno lavorativo del medico e non si sovrapponga temporalmente ad altre prestazioni dello stesso medico.

Attributi:

- `tipo` — tipologia della prestazione (visita o intervento chirurgico).
- `oraInizio` — ora di inizio della prestazione.
- `oraFine` — ora di fine della prestazione.
- `esito` — descrizione dell'esito della prestazione, modificabile dal medico.

Metodi:

- `checkSovrapposizione()` — verifica che la prestazione non si sovrapponga temporalmente a un'altra già eseguita dallo stesso medico. La traccia vieta esplicitamente la registrazione di prestazioni con sovrapposizione temporale per lo stesso medico.
-

3. Descrizione delle associazioni

3.1 Utente → Admin e Utente → Medico (Ereditarietà)

Si tratta di una relazione di generalizzazione. **Admin** e **Medico** sono sottoclassi di **Utente** ed ereditano i suoi attributi e metodi.

3.2 Medico → Reparto (Associazione 1..* a 1)

Un medico afferisce a esattamente un reparto, mentre un reparto ha almeno un medico

3.3 Medico → AssenzaMedico (Associazione 1 a 0..*)

Un medico può avere zero o più periodi di assenza per malattia registrati nel sistema. Zero perché nella normalità un medico non è assente; molti perché nel tempo può ammalarsi più volte.

3.4 Reparto → Stanza (Composizione 1 a 1..*)

È una relazione di composizione: una stanza non ha ragione di esistere al di fuori di un reparto. Se il reparto viene rimosso, le sue stanze non hanno più senso nel sistema. Un reparto contiene una o più stanze.

3.5 Stanza → Letto (Composizione 1 a 1..*)

Analogamente, un letto esiste solo all'interno di una stanza.

3.6 Paziente → Ricovero (Associazione 1 a 0..*)

Un paziente può avere zero o più ricoveri nel tempo. Zero perché un paziente può essere registrato in anagrafica senza essere mai stato ricoverato; molti perché può essere ricoverato più volte nel corso del tempo.

3.7 Letto → Ricovero (Associazione 1 a 0..*)

Un letto è associato a zero o più ricoveri nel tempo, ovviamente non sovrapposti.

3.8 Medico → Turno (Associazione 1 a 1..*)

Un medico ha almeno un turno lavorativo settimanale.

3.9 Ricovero → Prestazione (Associazione 1 a 0..*)

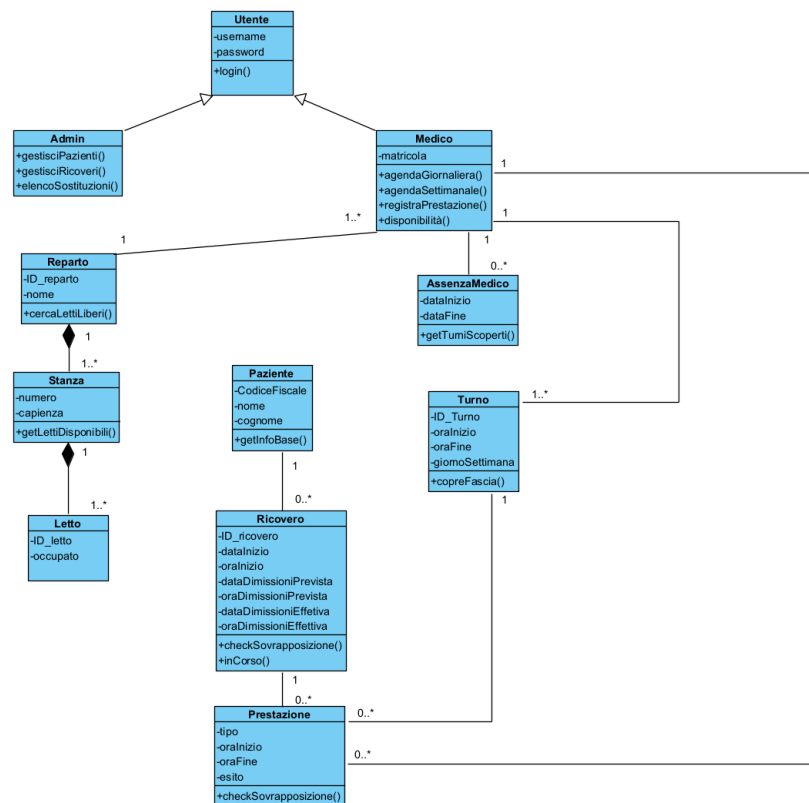
Durante un ricovero possono essere eseguite zero o più prestazioni. Zero perché è teoricamente possibile un ricovero senza prestazioni registrate; molti perché nel corso della degenza il medico può intervenire più volte.

3.10 Turno → Prestazione (Associazione 1 a 0..*)

Un turno può contenere zero o più prestazioni.

3.11 Medico → Prestazione (Associazione 1 a 0..*)

Un medico esegue zero o più prestazioni nel tempo.



4. Architettura di Sistema e Pattern BCE

L'architettura del sistema si basa sul pattern architetturale **BCE (Boundary-Control-Entity)**, una specializzazione del modello MVC. Questa scelta garantisce una rigorosa separazione delle responsabilità: il Model (Entity), descritto nelle sezioni precedenti, conserva i dati; la GUI (Boundary) gestisce esclusivamente l'interazione con l'utente; il Controller funge da coordinatore unico tra le due parti.

5. Descrizione del Controller

La logica di coordinamento è centralizzata in un'unica classe per semplificare la gestione dello stato dell'applicazione e assicurare che la GUI non acceda mai direttamente alle classi del Model.

5.1 Controller

Il **Controller** rappresenta il "motore" logico del sistema. Viene istanziato un'unica volta all'avvio del programma e il suo ciclo di vita coincide con quello dell'intera applicazione. Mantiene in memoria l'utente connesso e instrada le richieste della GUI verso i metodi specifici del Model.

- **Attributi:**
 - **utenteLoggato** – riferimento polimorfo (di tipo **Utente**) all'utente attualmente connesso. Permette di mantenere lo stato della sessione (Admin o Medico) durante le transizioni tra le varie schermate.
- **Metodi principali:**
 - **login()** – riceve le credenziali dalla GUI, verifica l'identità dell'utente (assegnando l'istanza corretta a **utenteLoggato**) e restituisce una stringa identificativa del ruolo ("admin", "medico" o "errore") per consentire alla GUI il routing verso il pannello corretto.
 - **logout()** – resetta lo stato della sessione annullando l'attributo **utenteLoggato**.
 - **Metodi di delega** (**gestisciPazienti()**, **agendaGiornaliera()**, **registraPrestazione()**, ecc.) – metodi operativi che vengono richiamati dai bottoni della GUI. Eseguono un downcasting sicuro di **utenteLoggato** per invocare le specifiche funzioni di business implementate nelle classi **Admin** e **Medico**.

6. Descrizione della GUI (Boundary)

Le classi del package `gui` si occupano esclusivamente della presentazione visiva tramite la libreria Java Swing. Nessuna elaborazione algoritmica dei dati è presente in queste classi; ogni interazione è delegata al `Controller`, che viene scambiato tra i vari pannelli tramite i costruttori per garantirne la persistenza e l'unicità.

6.1 Main

Rappresenta l'entry-point dell'applicativo.

- **Metodi:**
 - `main()` – configura il JFrame principale della finestra, istanzia l'oggetto unico Controller e inserisce nel frame il pannello radice del login invocando il metodo getter `.getMainPanel()` dell'oggetto `LoginPanel`, garantendo così il corretto caricamento dell'interfaccia grafica generata tramite lo Swing UI Designer .

6.2 LoginPanel

Rappresenta la schermata di autenticazione.

- **Attributi:**
 - `controller` – riferimento condiviso al controller dell'applicazione.
- **Metodi:**
 - Gestisce l'evento di click sul pulsante di login raccogliendo gli input testuali. Invoca l'omonimo metodo sul controller e, in base al ruolo restituito, estrae il JFrame radice per sostituirne dinamicamente il contenuto visivo con il pannello principale estratto tramite il metodo `.getMainPanel()` dalle nuove istanze di `AdminPanel` o `MedicoPanel`.

6.3 AdminPanel

Rappresenta la dashboard operativa riservata agli Amministratori.

- **Metodi:**
 - Inizializza l'interfaccia predisponendo i pulsanti dedicati alle funzionalità amministrative. Ad ogni evento di click, il pannello invoca il metodo corrispondente sul `Controller` condiviso. Implementa l'azione di *Logout*, che richiama la pulizia della sessione nel controller e ripristina la schermata `LoginPanel` garantendo l'aggiornamento grafico (tramite `revalidate()` e `repaint()`).

Rappresenta la dashboard operativa riservata al personale Medico.

- **Metodi:**

- Strutturata analogamente al pannello amministratore, espone i pulsanti per l'interazione con le agende, le prestazioni e la disponibilità. Anche in questa classe, l'evento di *Logout* gestisce la transizione grafica sicura verso la schermata di autenticazione preservando l'istanza univoca del controller.

